Souvik Paul

ic23btech11022@iith.ac.in | souvik05.paul@gmail.com | +91 8099083426 | LinkedIn | GitHub

EDUCATION

Institution	Degree / Board	Passing Year	Score / CGPA
Indian Institute of Technology Hyderabad	B.Tech	2027	8.60
South Point School	Class XII (CBSE)	2023	95.60%
South Point School	Class X (CBSE)	2021	97.83%

EXPERIENCE

Full-Stack Developer, NRIPelliGola

Backend & Infrastructure:

- Built **AI profile auto-fill**: users upload biodata as multi-page PDFs or images, **AWS Textract** extracts the text, and **Bedrock LLMs** map it to structured profile fields, cutting profile creation time from **~30 to ~10 minutes**.
- Replaced manual ID review with **automated approval** using Didit ID verification and **AWS Rekognition** face matching, removing a **~10 hr average wait** for verified users; added admin status, verification history, and VPN/IP fraud checks.
- Re-engineered **real-time chat** with auto-reconnect and offline recovery to fix silent message loss, and resolved 50+ message threads, read receipts, unseen counts, and iOS/mobile UX bugs.
- Shipped self-managed **web push notifications** (service workers, deep links, server-side validation), with **810 users (~33%)** opted in.
- Cut browse/shortlist API response time from **~9.8 s to under 1 s (~10x)** by eliminating redundant nested lookups, denormalizing frequently read data, and indexing common queries.
- Migrated user media (photos, verification videos) to **S3 served via CloudFront** (**85% cache hit ratio**) and fixed a browse memory overload.

Product & Growth:

- Built paid **1on1 calls** end to end: calendar integration, timezone-aware scheduling, confirmation emails, INR pricing, and an admin dashboard.
- Shipped **scratch-card rewards** (expiry, 12-hour reminder emails, server-side validation), lifting **conversion** from **53% to 57%** sustained (60% peak).

SKILLS

- **Languages:** C++, C, HTML, CSS, Javascript, SQL, Python, TypeScript
- **Technologies:** React.js, Node.js, Express.js, Next.js, PostgreSQL, TailwindCSS, Prisma ORM, OAuth, JWT Authentication, MongoDB, Websockets, Mongoose, AWS Textract, AWS Bedrock, AWS Rekognition
- **Tools:** Github, VSCode, Netlify, Vercel, Render, Supabase, Railway, Cloudinary, Socket.io, Redis, AWS, Jest, Pandas, Numpy, SciKit Learn, Matplotlib
- **Concepts and Technologies:** Data Structures and Algorithms, Database Management Systems, Object-Oriented Programming

PROJECTS

Real-Time Collaborative Workspace [Github]

- Built a **Notion-style collaborative editor** with Next.js 16, React 19, and TypeScript. **Multiple users edit the same document simultaneously** with live named cursors, using TipTap and Yjs CRDTs synced through a Hocuspocus WebSocket server.
- Engineered a **rich-text editing experience**: resizable tables and images **(drag-and-drop and paste upload)**, nested task lists, code blocks, custom font size/family/line-height/color controls, a floating formatting menu, and an **auto-generated table of contents** from document headings.
- Designed a Node.js/Express 5 backend that persists binary Yjs state to PostgreSQL via Prisma, using **debounced**

writes to cut database load, with **IndexedDB caching** on the client for offline resilience. Added a Socket.IO **per-workspace chat** with persisted message history.
- Implemented JWT authentication and **role-based access control** (owner / editor / viewer) across REST, WebSocket, and Socket.IO layers with Clerk, including read-only viewer connections and isolated personal/org workspaces. Backed by **Vitest/Supertest tests** covering auth and permission cases, run in CI on every push.

PgQ (Postgres Job Queue) [Github]

- Built **pgq**, a crash-safe job queue in TypeScript on PostgreSQL 16 (no Redis or broker), using SKIP LOCKED claiming, heartbeat leases, and lease fencing. Job completion and side effects commit in one transaction, giving **exactly-once effects** for same-database writes.
- Chaos-tested 30,000 jobs with **246 worker processes SIGKILLed** mid-job and SIGSTOP zombies: **0 jobs lost, 0 duplicate effects, 0 unexplained restarts**. Deliberately broken control workers were caught (7 duplicate effects, 356 unexplained restarts), proving the tests can fail.
- Cut claim query time from **37 ms to 0.5 ms (~75x)** using a partial composite index and query rewrite, with priority ordering adding no measurable throughput cost.
- Benchmarked on a 4-thread laptop: I/O-bound throughput scaled **26x** across workers, batched enqueue reached **32K jobs/s**, and claim latency held at **p99 under 13 ms**. Backed by 153 integration tests against a real Postgres.

RELEVANT COURSES

Programming Courses: Introduction to Programming in C language, Introduction to AI/ML, Machine Learning in Chemistry

Mathematics: Linear Algebra, Probability, Calculus

EXTRACURRICULAR

- Solved over 450+ problems across platforms like Leetcode, GeeksforGeeks, etc.

- Volunteered to organize and conduct learning programs for children in schools of villages, focusing on the inclusion of marginalized youth.